

Kedro

머신러닝 파이프라인

- 가상환경과 Kedro 설치

> Ubuntu Bash / 프로젝트를 만들 상위 폴더

- which python 결과가 kedro-class/.venv 아래인지 확인합니다.

```
mkdir -p ~/kedro-class
cd ~/kedro-class
python3.12 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install kedro

python --version
which python
git --version
kedro --version
```

- 새 프로젝트 생성과 패키지 설치

- > Ubuntu Bash / 같은 가상환경 유지

- 폴더 이름은 kedro-wine, Python 패키지 이름은 kedro_wine입니다.

```
kedro new --name "Kedro Wine" --tools none --example=n  
cd kedro-wine  
python -m pip install -e . --no-deps  
python -m pip install "kedro-datasets[pandas-csv,pandas-parquet,pickle,json]>=8,<9"  
python -m pip install "scikit-learn>=1.6,<1.7" "kedro-viz>=11.1,<12"  
python -m pip check  
kedro info
```

- 프로젝트에서 수정할 파일

- > `conf/base/catalog.yml`

- Dataset 이름, 저장 형식과 파일 경로 기록

- > `conf/base/parameters.yml`

- 분할 비율과 모델의 학습 조건 등 파라미터 기록

- > `src/kedro_wine/pipelines/`

- 각 모듈의 `nodes.py`와 `pipeline.py`

- > `src/kedro_wine/pipeline_registry.py`

- 명령으로 선택할 Pipeline 등록

- > `src/kedro_wine/hooks.py`와 `settings.py`

- MLflow 기록 동작과 Hook 등록

- 실습 데이터 생성

> 프로젝트 루트에 prepare_data.py 작성

- 실행: python prepare_data.py / 예상 크기: (178, 15)

```
from pathlib import Path
from sklearn.datasets import load_wine

data = load_wine(as_frame=True).frame
data.insert(0, "row_id", range(len(data)))
path = Path("data/01_raw/wine.csv")
path.parent.mkdir(parents=True, exist_ok=True)
data.to_csv(path, index=False)
print(data.shape)
print(data["target"].value_counts().sort_index())
```

- 체크포인트 1

- > 프로젝트와 데이터가 준비되었는가?
- > 현재 폴더에 pyproject.toml이 있다.
- > kedro info가 프로젝트를 인식한다.
- > data/01_raw/wine.csv에 178행의 데이터가 있다.
- > row_id와 target을 학습 특성에 넣지 않는 이유를 설명한다.
 - 다음 단계: CSV 파일에 wine_raw라는 Dataset 이름을 붙입니다.

- Data Catalog의 역할

- > 논리적인 이름과 실제 입출력을 연결

- > Node에서 사용하는 이름

- wine_raw, train_data, model

- > Catalog에 기록하는 정보

- type은 읽고 저장하는 구현, filepath는 실제 위치

- > 실행 시 Kedro가 하는 일

- 입력 이름으로 load한 값을 함수에 전달하고 반환값을 save

- > 함수에서 다루는 값

- DataFrame, 학습된 모델, 지표 dict처럼 이미 읽힌 Python 객체
- 파일 경로가 바뀌면 우선 Catalog를 수정합니다.

- 원본 CSV 등록

> conf/base/catalog.yml / 새로 작성

- CSVDataset의 대소문자와 YAML의 공백 들여쓰기를 확인합니다.

```
wine_raw:
  type: pandas.CSVDataset
  filepath: data/01_raw/wine.csv
  load_args:
    encoding: utf-8
  metadata:
    kedro-viz:
      layer: raw
```

- 분할 데이터 저장 설정

> 같은 catalog.yml 아래에 추가

- 세 파일은 분할 Node가 실행된 후에 생깁니다.

train_data:

type: pandas.ParquetDataset

filepath: data/05_model_input/train.parquet

val_data:

type: pandas.ParquetDataset

filepath: data/05_model_input/val.parquet

test_data:

type: pandas.ParquetDataset

filepath: data/05_model_input/test.parquet

• 모델과 검증 결과 저장 설정 (1/2)

> 같은 catalog.yml 아래에 추가

- 이 기본 설정은 재실행할 때 같은 경로의 결과를 덮어씁니다.

model:

type: pickle.PickleDataset

filepath: data/06_models/model.pkl

metrics:

type: json.JSONDataset

filepath: data/08_reporting/metrics.json

- 모델과 검증 결과 저장 설정 (2/2)

> 같은 catalog.yml 아래에 추가

- 이 기본 설정은 재실행할 때 같은 경로의 결과를 덮어씁니다.

```
predictions:  
  type: pandas.CSVDataset  
  filepath: data/07_model_output/predictions.csv  
  save_args:  
    index: false
```

- 실행 조건을 파라미터로 분리

> conf/base/parameters.yml / 아래 내용으로 작성

- Node 입력에서 params:split_options, params:model_options로 참조합니다.

```
split_options:
```

```
  test_size: 0.2
```

```
  val_size: 0.25
```

```
  seed: 42
```

```
model_options:
```

```
  C: 1.0
```

```
  max_iter: 2000
```

```
  seed: 42
```

• 체크포인트 2

> 설정과 실제 파일의 관계

> 등록된 Dataset은 7개

- wine_raw, train_data, val_data, test_data, model, metrics, predictions

> validated_data는 등록하지 않음

- 한 번의 실행 안에서만 쓰는 중간 출력은 MemoryDataset으로 전달

> params: 접두사 확인

- 파라미터 이름을 일반 Dataset 이름과 구분

> 현재 존재해야 하는 데이터 파일

- wine.csv 한 개, 나머지 결과는 Pipeline 실행 후 생성
- 질문: 파일 저장 없이 다음 프로세스에서 validated_data를 읽을 수 있을까요?

- Python 함수와 Node의 연결

> Node는 함수에 입출력 Dataset 이름을 붙임

- inputs의 문자열은 Python 변수명이 아니라 Dataset 이름입니다.

```
from kedro.pipeline import node

def add(a, b):
    return a + b

n = node(add, inputs=["left", "right"],
          outputs="total", name="add_numbers")
print(n.run({"left": 2, "right": 3}))
# {"total": 5}
```

- Node 입력 이름과 함수 인수

- > 위치 인수와 키워드 인수의 연결

- Dataset 이름 left·right와 함수 인수 a·b를 구분합니다.

```
# 순서대로 함수 인수에 전달
```

```
n1 = node(add, ["left", "right"], "total")
```

```
# 함수 인수 이름을 명시해 전달
```

```
n2 = node(add, {"a": "left", "b": "right"}, "total")
```

```
print(n2.run({"left": 2, "right": 3}))
```

```
# {"total": 5}
```

- 한 Node에서 여러 출력 반환

> 반환 개수와 Dataset 이름의 개수 일치

- 분할 함수도 train·val·test 세 Dataset을 반환합니다.

```
def extremes(values):  
    return min(values), max(values)  
  
n = node(extremes, "values", ["minimum", "maximum"],  
         name="find_extremes")  
print(n.run({"values": [2, 5, 9]}))  
# {"minimum": 2, "maximum": 9}
```

- 두 개의 모듈 생성

> Ubuntu Bash / 프로젝트 루트

- 이후 제시하는 코드를 각 파일에 작성합니다. 빈 예제 코드는 교체합니다.

```
# 모듈 생성
```

```
kedro pipeline create data_processing
```

```
kedro pipeline create data_science
```

- 두 개의 모듈 생성 / 확인

- > Ubuntu Bash / 프로젝트 루트

- > data_processing

- nodes.py에 검증·분할 함수, pipeline.py에 연결 정의

- > data_science

- nodes.py에 학습·평가 함수, pipeline.py에 연결 정의
- 이후 제시하는 코드를 각 파일에 작성합니다. 빈 예제 코드는 교체합니다.

- 데이터 검증 함수

> src/kedro_wine/pipelines/data_processing/nodes.py / 앞부분

- 경로를 받아 CSV를 읽지 않고, DataFrame을 받아 검사합니다.

```
from sklearn.model_selection import train_test_split

def validate_data(data):
    required = {"row_id", "target"}
    if not required.issubset(data.columns):
        raise ValueError("Missing required columns")
    if data.empty or data.isna().any().any():
        raise ValueError("Empty data or missing values")
    if not data["row_id"].is_unique:
        raise ValueError("Duplicate row_id")
    return data.copy()
```

- 학습·검증·테스트 분할

> 같은 data_processing/nodes.py 아래에 추가

- 반환 순서 train, val, test를 Node의 outputs 순서와 맞춥니다.

```
def split_data(data, options):  
    dev, test = train_test_split(  
        data, test_size=options["test_size"],  
        random_state=options["seed"],  
        stratify=data["target"],  
    )  
    train, val = train_test_split(  
        dev, test_size=options["val_size"],  
        random_state=options["seed"],  
        stratify=dev["target"],  
    )  
    return train, val, test
```

- 전처리와 분류 모델 학습 (1/2)

> src/kedro_wine/pipelines/data_science/nodes.py / 앞부분

- SkPipeline은 모델 내부 전처리이며 Kedro Pipeline과 역할이 다릅니다.

```
from sklearn.pipeline import Pipeline as SkPipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, f1_score
```

• 전처리와 분류 모델 학습 (2/2)

> src/kedro_wine/pipelines/data_science/nodes.py / 앞부분

- SkPipeline은 모델 내부 전처리이며 Kedro Pipeline과 역할이 다릅니다.

```
def train_model(train_data, options):  
    model = SkPipeline([  
        ("scale", StandardScaler()),  
        ("classifier", LogisticRegression(  
            C=options["C"], max_iter=options["max_iter"],  
            random_state=options["seed"])),  
    ])  
    X = train_data.drop(columns=["row_id", "target"])  
    return model.fit(X, train_data["target"])
```

- 검증 지표와 행별 예측

> 같은 `data_science/nodes.py` 아래에 추가

- 지표 dict와 예측 DataFrame을 각각 다른 Dataset에 저장합니다.

```
def evaluate_model(model, data):
    X = data.drop(columns=["row_id", "target"])
    pred = model.predict(X)
    metrics = {
        "accuracy": float(accuracy_score(data["target"], pred)),
        "f1_macro": float(f1_score(
            data["target"], pred, average="macro")),
    }
    predictions = data[["row_id", "target"]].copy()
    predictions["prediction"] = pred
    return metrics, predictions
```

• 데이터 준비 Pipeline

> src/kedro_wine/pipelines/data_processing/pipeline.py / 전체

- validated_data라는 동일한 이름이 두 Node의 의존성을 만듭니다.

```
from kedro.pipeline import Pipeline, node
from .nodes import validate_data, split_data

def create_pipeline(**kwargs):
    return Pipeline([
        node(validate_data, "wine_raw", "validated_data",
            name="validate_data", tags=["prepare"]),
        node(split_data,
            ["validated_data", "params:split_options"],
            ["train_data", "val_data", "test_data"],
            name="split_data", tags=["prepare"]),
    ])
```

• 학습·평가 Pipeline

> src/kedro_wine/pipelines/data_science/pipeline.py / 전체

- 이 Pipeline은 test_data를 사용하지 않습니다.

```
from kedro.pipeline import Pipeline, node
from .nodes import train_model, evaluate_model

def create_pipeline(**kwargs):
    return Pipeline([
        node(train_model,
            ["train_data", "params:model_options"], "model",
            name="train_model", tags=["train"]),
        node(evaluate_model, ["model", "val_data"],
            ["metrics", "predictions"],
            name="evaluate_model", tags=["eval"]),
    ])
```

- CLI에서 실행할 Pipeline 등록

> src/kedro_wine/pipeline_registry.py / 전체

- kedro run은 __default__, --pipeline 옵션은 등록된 이름을 선택합니다.

```
from kedro_wine.pipelines import data_processing as dp
from kedro_wine.pipelines import data_science as ds
```

```
def register_pipelines():
    prepare = dp.create_pipeline()
    science = ds.create_pipeline()
    return {
        "__default__": prepare + science,
        "data_processing": prepare,
        "data_science": science,
    }
```

- 전체 실행과 결과 확인

> Ubuntu Bash / 프로젝트 루트

- 명령 종료뿐 아니라 입력·출력 이름과 저장 위치를 확인합니다.

```
kedro run  
cat data/08_reporting/metrics.json  
head -n 4 data/07_model_output/predictions.csv  
ls data/05_model_input  
ls data/06_models
```

- 전체 실행과 결과 확인 / 확인

- > Ubuntu Bash / 프로젝트 루트

- > 실행 로그에서 네 Node 확인

- validate_data, split_data, train_model, evaluate_model

- > 결과 파일 확인

- 분할 Parquet 3개, model.pkl, metrics.json, predictions.csv
 - 명령 종료뿐 아니라 입력·출력 이름과 저장 위치를 확인합니다.

- 부분 실행과 입력 경계

- > 전체 실행이 한 번 성공한 후 비교

- 부분 실행은 선택한 범위의 외부 입력을 자동으로 만들어 주지 않습니다.

```
kedro run --pipeline data_science
kedro run --nodes train_model
kedro run --from-nodes train_model
kedro run --to-nodes split_data
```

명령어	의미
<code>kedro run --pipeline</code>	이름으로 등록된 파이프라인 하나를 통째로 실행
<code>kedro run --nodes</code>	지정한 노드만 실행 (앞뒤 노드는 실행 안 함)
<code>kedro run --from-nodes</code>	지정한 노드부터 끝까지 실행 (그 노드와 모든 후속 노드)
<code>kedro run --to-nodes</code>	처음부터 지정한 노드까지 실행 (그 노드와 필요한 모든 선행 노드)

- 부분 실행과 입력 경계 / 확인

- > 전체 실행이 한 번 성공한 후 비교

- > 파일로 남긴 `train_data`와 `val_data`

- 새 프로세스에서 학습·평가만 실행할 수 있음

- > 메모리에만 둔 `validated_data`

- `--from-nodes split_data`만으로는 이전 실행 값을 복원할 수 없음
- 부분 실행은 선택한 범위의 외부 입력을 자동으로 만들어 주지 않습니다.

• 체크포인트 3

- > 코드 순서보다 데이터 이름으로 설명
- > 각 Node의 입력과 출력을 한 번씩 읽어 본다.
- > Node 목록에서 학습을 앞에 적어도 분할이 먼저 실행되는 이유를 설명한다.
- > train_data가 106행, val_data와 test_data가 각각 36행인지 확인한다.
- > model.pkl에 스케일러와 분류기가 함께 저장되는지 설명한다.
 - 다음 단계: 코드에서 정의한 연결을 Kedro-Viz에서 직접 확인합니다.

- Kedro-Viz 실행

> Ubuntu Bash / 새 터미널에서 같은 환경과 프로젝트 선택

- 서버와 브라우저가 서로 다른 컴퓨터에 있으면 포트 전달이 필요합니다.

```
cd kedro-wine
```

```
kedro viz run --host 127.0.0.1 --port 4141
```

- Kedro-Viz 실행 / 확인

- > Ubuntu Bash / 새 터미널에서 같은 환경과 프로젝트 선택

- > 브라우저 주소

- <http://127.0.0.1:4141>

- > 터미널 사용

- 서버 실행 상태를 유지하고 종료할 때 Ctrl+C
- 서버와 브라우저가 서로 다른 컴퓨터에 있으면 포트 전달이 필요합니다.

- 화면에서 찾을 네 개의 Node

- > `__default__` Pipeline을 선택하고 입력·출력을 추적

- > `validate_data`

- 입력 `wine_raw` / 출력 `validated_data`

- > `split_data`

- 입력 `validated_data`와 `split_options` / 출력 세 분할 데이터

- > `train_model`

- 입력 `train_data`와 `model_options` / 출력 `model`

- > `evaluate_model`

- 입력 `model`과 `val_data` / 출력 `metrics`와 `predictions`
- `test_data`가 학습·검증 평가로 연결되지 않는지 확인합니다.

- 레이어와 그래프 오류 확인

> Catalog의 metadata로 데이터 단계를 구분

- 변경 후 Viz를 재시작하거나 --autoreload로 실행합니다.

```
# 기존 train_data 항목 아래에 추가
```

```
metadata:
```

```
  kedro-viz:
```

```
    layer: model_input
```

- 레이어와 그래프 오류 확인 / 확인

- > Catalog의 metadata로 데이터 단계를 구분
- > wine_raw는 raw, 분할 데이터는 model_input으로 구분
- > 연결이 이상하면 inputs / outputs의 문자열부터 비교
- > 그래프가 안 뜨면 import 오류와 Pipeline Registry를 확인
 - 변경 후 Viz를 재시작하거나 --autoreload로 실행합니다.

• 체크포인트 4

- > 화면으로 설명하는 데이터 의존성
- > wine_raw에서 metrics까지 이어지는 경로를 찾는다.
- > 학습 함수가 val_data를 사용하지 않는지 확인한다.
- > split_data를 제거했을 때 학습에 필요한 외부 입력을 찾는다.
- > 그래프의 선과 Pipeline 코드의 Dataset 이름을 대응시킨다.
 - Kedro-Viz는 실행 구조를, 다음 단계의 MLflow는 실험 결과를 보여줍니다.

- Kedro 실행과 MLflow 실험 기록

- > 이 수업은 기존 자료의 직접 Hook 방식을 사용

- > Kedro

- 어떤 데이터와 함수가 실행에 필요한지 관리

- > Kedro-Viz

- 입출력 관계와 Pipeline 구조를 시각화

- > MLflow

- 실행별 파라미터, 지표와 모델을 기록하고 비교

- > 기록 단위

- 추적을 켜 kedro run 한 번에 MLflow Run 한 개
- kedro-mlflow 플러그인은 보충 자료에서 대안으로 다룹니다.

- MLflow 설치와 서버 시작

> Ubuntu Bash / 새 터미널, 같은 가상환경과 프로젝트 루트

- 이 터미널은 유지하고 학습은 다른 터미널에서 실행합니다.

```
python -m pip install "mlflow>=3,<4"
```

```
python -m pip check
```

```
mlflow server --host 127.0.0.1 --port 5000 ₩
```

```
--backend-store-uri sqlite:///mlflow.db ₩
```

```
--serve-artifacts --artifacts-destination ./artifacts
```

- Hook으로 기록할 시점

- > 학습 함수는 그대로 유지

- > `before_pipeline_run`

- Tracking URI 설정, 실험 선택, Run 시작

- > `after_node_run`

- 실제로 전달된 파라미터, 계산한 검증 지표, 학습 모델 기록

- > `after_pipeline_run`

- 정상 완료한 Run 종료

- > `on_pipeline_error`

- 실패한 Run 종료
- 다음 네 슬라이드의 코드를 하나의 `hooks.py`에 순서대로 작성합니다.

- Hook 1 / Run 시작 (1/2)

> src/kedro_wine/hooks.py / 새 파일의 시작

- MLFLOW_TRACKING_URI를 설정한 실행에만 추적을 적용합니다.

```
import os
import mlflow
import mlflow.sklearn
from kedro.framework.hooks import hook_impl

class TrackingHooks:
    def __init__(self):
        self.owns_run = False
```

- Hook 1 / Run 시작 (2/2)

> src/kedro_wine/hooks.py / 새 파일의 시작

- MLFLOW_TRACKING_URI를 설정한 실행에만 추적을 적용합니다.

```
@hook_impl
def before_pipeline_run(self, run_params):
    uri = os.getenv("MLFLOW_TRACKING_URI")
    if not uri:
        return
    mlflow.set_tracking_uri(uri)
    mlflow.set_experiment("kedro-wine-class")
    mlflow.start_run(run_name=run_params.get("pipeline_name") or "default")
    self.owns_run = True
```

• Hook 2 / 파라미터와 지표 기록 (1/2)

> 같은 TrackingHooks 클래스 아래에 추가

- 이 실습의 두 파라미터는 dict이며 키를 붙여 충돌을 피합니다.

```
@hook_impl
def after_node_run(self, node, inputs, outputs):
    if not self.owns_run:
        return
    for name, value in inputs.items():
        if name.startswith("params:"):
            prefix = name.removeprefix("params:")
```

• Hook 2 / 파라미터와 지표 기록 (2/2)

> 같은 TrackingHooks 클래스 아래에 추가

- 이 실습의 두 파라미터는 dict이며 키를 붙여 충돌을 피합니다.

```
        mlflow.log_params({
            f"{prefix}.{k}": v for k, v in value.items()
        })
    if "metrics" in outputs:
        mlflow.log_metrics(outputs["metrics"])
    if node.name == "train_model":
        self.log_model(inputs, outputs)
```

• Hook 3 / 모델과 입력 예시 기록

> 같은 TrackingHooks 클래스 아래에 추가

- MLflow 3 기준: input_example을 사용해 모델 입력·출력 signature를 추론합니다.

```
def log_model(self, inputs, outputs):
    X = inputs["train_data"].drop(
        columns=["row_id", "target"]
    ).head(5)
    mlflow.sklearn.log_model(
        outputs["model"],
        name="model",
        input_example=X,
    )
```

• Hook 4 / 정상·실패 종료 (1/2)

> 같은 TrackingHooks 클래스 아래에 추가

- 자신이 시작한 Run만 종료하도록 owns_run을 확인합니다.

```
def finish(self, status):  
    if self.owns_run:  
        try:  
            mlflow.end_run(status=status)  
        finally:  
            self.owns_run = False
```

```
@hook_impl  
def after_pipeline_run(self):  
    self.finish("FINISHED")
```

- Hook 4 / 정상·실패 종료 (2/2)

> 같은 TrackingHooks 클래스 아래에 추가

- 자신이 시작한 Run만 종료하도록 owns_run을 확인합니다.

```
@hook_impl
def on_pipeline_error(self):
    self.finish("FAILED")
```

- Hook 등록과 추적 활성화

> src/kedro_wine/settings.py에 추가

- settings.py에 HOOKS가 이미 있으면 기존 튜플에 인스턴스를 추가합니다.

```
from kedro_wine.hooks import TrackingHooks
```

```
HOOKS = (TrackingHooks(),)
```

- Hook 등록과 추적 활성화 / 확인

- > src/kedro_wine/settings.py에 추가

- > 학습용 Ubuntu Bash 터미널에서 실행

- export MLFLOW_TRACKING_URI=http://127.0.0.1:5000

- > 추적 실행

- kedro run

- > 다시 추적을 끌 때

- unset MLFLOW_TRACKING_URI
- settings.py에 HOOKS가 이미 있으면 기존 튜플에 인스턴스를 추가합니다.

- C 값을 바꾼 두 번의 실험

- > 원본·분할 seed를 유지하고 전체 Pipeline을 실행

- metrics.json은 마지막 실행 값입니다. 이력 비교는 MLflow에서 합니다.

```
kedro run --params model_options.C=0.1  
kedro run --params model_options.C=1.0
```

- C 값을 바꾼 두 번의 실험 / 확인

- > 원본·분할 seed를 유지하고 전체 Pipeline을 실행

- > 변경하는 값

- LogisticRegression의 C만 변경

- > 유지하는 조건

- wine.csv, split_options, seed, 평가 지표

- > 예상 결과

- MLflow에 서로 다른 Run 두 개와 각각의 모델 기록
- metrics.json은 마지막 실행 값입니다. 이력 비교는 MLflow에서 합니다.

- MLflow 화면에서 실험 비교 / 확인
 - > Windows PowerShell / MLflow용 SSH 터널
 - > `http://127.0.0.1:5000`에서 `kedro-wine-class` 선택
 - > 두 Run의 `model_options.C`, `accuracy`, `f1_macro` 비교
 - > 각 Run의 모델 상세에서 입력 예시와 `signature` 확인
 - > Run ID와 선택한 C를 함께 기록
 - 서버 시작 위치와 Tracking URI가 다르면 다른 실험 저장소를 볼 수 있습니다.

• 체크포인트 5

- > 실험 기록이 재현 조건을 설명하는가?
- > 각 Run의 상태가 FINISHED이며 지표 두 개가 있다.
- > 두 Run에서 model_options.C가 서로 다르다.
- > 기록한 모델의 입력에 row_id와 target이 없다.
- > 선택한 Run ID, C, 검증 macro F1을 말할 수 있다.
- > 테스트 데이터는 아직 후보 선택에 사용하지 않았다.
 - 최종 테스트는 후보를 확정된 뒤 한 번만 수행합니다.

• 다섯 단계 복습

> 프로젝트 생성

- 명령을 실행할 환경과 폴더 구조 준비

> Data Catalog

- 입출력 이름에 형식과 저장 위치 연결

> Node와 Pipeline

- 함수의 입력·출력을 연결하고 실행 단위를 등록

> Kedro-Viz

- 데이터 의존성과 학습·검증 경계 확인

> MLflow

- 실행별 조건·지표·모델을 기록하고 비교

- 선택한 모델의 최종 테스트 (1/2)

- > 선택한 C로 다시 실행한 뒤 프로젝트 루트에 final_test.py 작성

- 선택 C가 0.1이면 `kedro run --params model_options.C=0.1` 실행 후 `python final_test.py`

```
import json
import pickle
from pathlib import Path
import pandas as pd
from kedro_wine.pipelines.data_science.nodes import evaluate_model
```

• 선택한 모델의 최종 테스트 (2/2)

> 선택한 C로 다시 실행한 뒤 프로젝트 루트에 final_test.py 작성

- 선택 C가 0.1이면 `kedro run --params model_options.C=0.1` 실행 후 `python final_test.py`

```
with open("data/06_models/model.pkl", "rb") as f:
    model = pickle.load(f)
test = pd.read_parquet("data/05_model_input/test.parquet")
metrics, predictions = evaluate_model(model, test)
Path("data/08_reporting/test_metrics.json").write_text(
    json.dumps(metrics, indent=2), encoding="utf-8")
predictions.to_csv("data/07_model_output/test_predictions.csv", index=False)
print(metrics)
```

• 자주 만나는 오류

> kedro 명령 또는 모듈을 찾지 못함

- 가상환경 활성화, which python, pip check 확인

> Dataset 또는 파일을 찾지 못함

- Catalog 키, 입력 이름, 프로젝트 경로, 선행 저장 파일 확인

> Pipeline을 만들 수 없음

- 중복 출력, 순환 의존성, import와 Registry 확인

> Viz 또는 MLflow 화면에 접속하지 못함

- 서버 터미널, 실제 포트, SSH 터널 유지 여부 확인

> Kedro는 성공했지만 MLflow Run이 없음

- 학습 터미널의 Tracking URI와 settings.py의 HOOKS 확인